INSTITUTO FEDERAL CATARINENSE - IFC
BACHARELADO DE CIÊNCIA DA COMPUTAÇÃO

ANDRÉ VITOR BASTOS DE MACÊDO

ALGORITMOS DE BUSCA DE CAMINHO EM REPRESENTAÇÕES NAVEGÁVEIS TRIDIMENSIONAIS : Uma análise estatística e comparativa

Blumenau
2026

ANDRÉ VITOR BASTOS DE MACÊDO

ALGORITMOS DE BUSCA DE CAMINHO EM REPRESENTAÇÕES NAVEGÁVEIS TRIDIMENSIONAIS : Uma análise estatística e comparativa

Trabalho de Conclusão de Curso apresentado ao curso de Bacharelado em Ciência da Computação do Instituto Federal Catarinense.

Orientador(a): Prof. Paulo César Rodacki Gomes

Blumenau
2026

LISTA DE ILUSTRAÇÕES

Figura 1 Comparação entre representações 2D e 3D 7
Figura 2 Exemplo de grafo simples. 9
Figura 3 Exemplo de grafo direcionado e não direcionado. 11
Figura 4 Exemplo de grafo ponderado e não ponderado. 12
Figura 5 Exemplo de caminho em um grafo. 13
Figura 6 Exemplo de componentes conexos em um grafo. 13
Figura 7 Diversos caminhos possíveis entre dois vértices em um grafo. 14
Figura 8 Pseudocódigo do algoritmo de Dijkstra. 14
Figura 9 Pseudocódigo do algoritmo A*. 17
Figura 10 Pseudocódigo do algoritmo Weighted A*. 18
Figura 11 Pseudocódigo da Greedy Best-First Search. 19
Figura 12 Conversão de um mapa de altura em vértices tridimensionais. 20
Figura 13 Exemplo de malha gerada a partir de um mapa de altura 20
Figura 14 Ruído de Perlin com células de diferentes tamanhos. 21
Figura 15 Variação da lacunaridade: ruídos com lacunaridade baixa e alta. 23
Figura 16 Variação da persistência: ruídos com persistência baixa e alta. 23
Figura 17 Variação do número de oitavas: ruídos com 1 e 6 oitavas. 24
Figura 18 Exemplo de reconstrução de superfícies com Marching Cubes e Transvoxel. 25
Figura 19 Exemplo de grade 2.5D. 25
Figura 20 Exemplo de superfície voxel. 26
Figura 21 Exemplo de grafo de vértices. 26
Figura 22 Exemplo de grafo dual. 26

LISTA DE TABELAS

LISTA DE ABREVIATURAS E SIGLAS

SIG Sigla

SUMÁRIO

1 INTRODUÇÃO 7
1.1 Contribuições 8
1.2 Objetivos 8
2 FUNDAMENTAÇÃO TEÓRICA 9
2.1 Grafos 9
2.1.1 Definição 9
2.1.2 Direcionamento 11
2.1.3 Pesos 11
2.1.4 Caminho 12
2.1.5 Componente conexo 13
2.2 Busca 14
2.2.1 Dijkstra 14
2.2.2 Heurísticas 15
2.2.3 A* 16
2.2.4 Weighted A* 17
2.2.5 Greedy Best-First Search 18
2.3 Geração Procedural de Terrenos 19
2.3.1 Mapas de altura 19
2.3.2 Ruído de Perlin 21
2.4 Representações Geométricas de Terrenos 24
2.4.1 Voxels 24
2.4.2 Marching Cubes/Transvoxel 25
2.5 Representações Navegáveis de Terrenos 25
2.5.1 Grade 2.5D 25
2.5.2 Superfície Voxel 26
2.5.3 Grafo de Vértices 26
2.5.4 Grafo Dual 26
2.6 Avaliação de desempenho e estatísticas 26
2.7 Trabalhos relacionados 27
2.7.1 Johansson 27
2.7.2 Kapi 27
2.7.3 Matriz de diferença 27
3 DESENVOLVIMENTO 28
3.1 Pipeline 28
3.2 IFCG e visualizador concorrente 28
3.3 lwGraph<T> 28
3.4 search.hpp 28
3.5 Gerador de terrenos 28
3.6 GraphModel, GraphFactory, isTranversable 28
3.7 Instrumentação 28

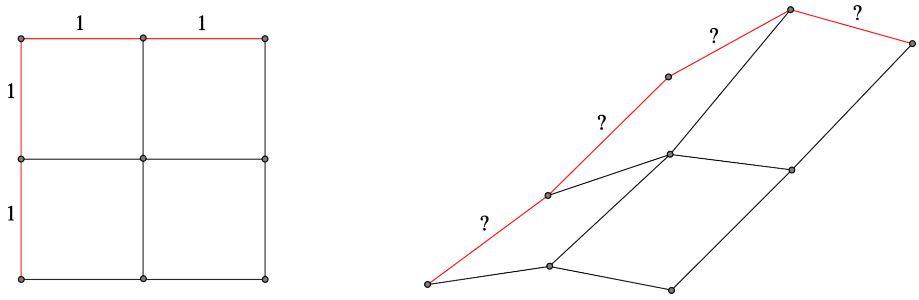
3.7.1 Statistics 28
3.7.2 acquire.sh 29
3.7.3 CLI 29
4 METODOLOGIA 30
4.1 Tipo de pesquisa 30
4.2 Ambiente 30
4.3 Variáveis 30
4.4 Níveis 30
4.5 Procedimento e métricas 30
4.6 Protocolo estatístico 30
4.7 Ameaças 30
5 RESULTADOS E DISCUSSÃO 31
6 CONCLUSÃO 32
REFERÊNCIAS 33

1 INTRODUÇÃO

A busca de caminhos (ou *pathfinding*) é um dos campos mais tradicionais da Inteligência Artificial aplicada, evoluindo a partir dos algoritmos de busca clássicos em grafos (ver Seção 2.1). Como definem Russell; Norvig (2013), os métodos clássicos de busca resolvem problemas de estados, ações e custos, visando encontrar caminhos entre um ponto de partida e um objetivo por meio da exploração do espaço de busca. Ao longo das últimas décadas, essa fundamentação deixou de ser puramente abstrata e passou a ser aplicada diretamente na computação interativa. Conforme apontam Pardede et al. (2022), os algoritmos de busca tornaram-se componentes cruciais para o desenvolvimento de jogos modernos e simulações em tempo real.

No entanto, a transição do espaço de busca de 2D para 3D introduziu um aumento significativo na complexidade, exigindo adaptações às técnicas de busca e de representação do terreno. A escolha da representação do espaço transitável e do algoritmo de busca a ser utilizado tornou-se uma decisão crítica, impactando diretamente o desempenho e a qualidade dos caminhos encontrados.

Figura 1 – Comparação entre representações 2D e 3D



Fonte: Elaborado pelo autor.

Em estudos recentes, é comumente evitada a comparação de múltiplas representações navegáveis de terrenos 3D, com foco em apenas uma abordagem na maioria das vezes. Além disso, a análise estatística do desempenho dos algoritmos de busca em diferentes representações é frequentemente negligenciada, limitando a compreensão das interações entre a estrutura do terreno e o comportamento dos algoritmos.

Esse trabalho propõe uma análise comparativa e estatística do desempenho de algoritmos clássicos de busca de caminhos em diferentes representações navegáveis de terrenos 3D. A pesquisa busca responder à seguinte questão central: Qual é a influência da **representação navegável** de um terreno 3D e das **características do terreno** no desempenho e na qualidade da busca de algoritmos clássicos de busca?

1.1 Contribuições

Para responder à questão central, é necessário realizar uma análise detalhada das representações navegáveis de terrenos 3D (ver Seção 2.5), dos algoritmos de busca aplicados (ver Seção 2.2) e das métricas de desempenho utilizadas (ver Seção 2.6). Com isso, as principais contribuições deste trabalho incluem:

1. **Comparação entre representações navegáveis.**
   As mesmas instâncias de terreno são convertidas em quatro grafos diferentes e avaliadas com os mesmos algoritmos:
   - a partir do mapa de altura: grade de altura (2.5D) e superfície voxel;
   - a partir da malha poligonal: grafo de vértices e grafo dual.
2. **Metodologia experimental reprodutível.**
   Os terrenos são gerados proceduralmente (ver Seção 2.3) a partir de parâmetros controlados (escala, lacunaridade, persistência) e semente registrada, e os resultados passam por análise estatística (análise de variância, ou ANOVA, e testes post hoc).
3. **Implementação open-source.**
   Inclui uma biblioteca de grafos leve e o motor gráfico IFCG, com visualização das malhas e dos caminhos.

1.2 Objetivos

O objetivo geral do estudo é analisar comparativa e estatisticamente o desempenho de algoritmos clássicos de *pathfinding* em diferentes representações navegáveis de terrenos 3D. Mais especificamente, os objetivos incluem:

1. Implementar uma biblioteca de grafos e os algoritmos Dijkstra, A*, Weighted A* e Greedy Best-First Search.
2. Desenvolver um gerador procedural de terrenos baseado em ruído de Perlin, parametrizado por escala, lacunaridade e persistência.
3. Construir quatro representações navegáveis a partir do terreno: grade de altura, superfície voxel, grafo de vértices e grafo dual da malha.
4. Desenvolver um ambiente 3D com execução concorrente para visualização das malhas e dos caminhos encontrados.
5. Automatizar a coleta de métricas e analisar estatisticamente os resultados obtidos.

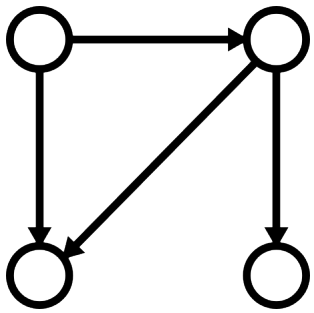
2 FUNDAMENTAÇÃO TEÓRICA

A comparação proposta neste trabalho depende de três conjuntos de conceitos: a teoria de grafos, que modela o espaço de busca; os algoritmos que percorrem esse modelo; e os métodos de representação de terrenos que determinam como o grafo é construído a partir da geometria. Este capítulo apresenta cada um deles, nessa ordem, e termina com os trabalhos que compararam representações semelhantes.

2.1 Grafos

Grafos são estruturas matemáticas que representam relações entre objetos. Eles consistem em um conjunto de vértices (ou nós) e um conjunto de arestas (ou ligações) que conectam os vértices. Um grafo pode ser direcionado ou não direcionado, ponderado ou não ponderado, dependendo das características do problema a ser modelado.

Figura 2 – Exemplo de grafo simples.



Fonte: Elaborado pelo autor.

Quando trabalhamos com estruturas de dados complexas, como terrenos 3D, os grafos fornecem uma representação abstrata que facilita a aplicação de algoritmos de busca. Em aplicações como a navegação em terrenos 3D, os vértices podem representar posições do ambiente, e as arestas, a transitabilidade entre elas.

2.1.1 Definição

Segundo Gomes (2022), um grafo G é definido como um par ordenado (Equação 1), onde: G é o grafo; V é o conjunto de vértices; e A é o conjunto de arestas. Cada aresta representa a conexão entre dois vértices, podendo ter uma direção associada e um peso.

$$G = (V, A) \tag{1}$$

Cada conjunto de vértices e arestas pode ser representado de diferentes maneiras, dependendo da aplicação e das operações que serão realizadas sobre o grafo. As representações mais comuns incluem listas de adjacência (Seção 2.1.1.3) e matrizes de adjacência (Seção 2.1.1.4) (Gomes, 2022).

Dado que um grafo é uma estrutura de dados baseada em conjuntos, naturalmente existem subgrafos. Um subgrafo é formado por um subconjunto de vértices e por um subconjunto de arestas cujas extremidades pertencem a esse subconjunto de vértices, podendo ser utilizado para analisar partes específicas da estrutura ou para simplificar a resolução de problemas complexos (Gomes, 2022).

2.1.1.1 Vértices

Vértices são as unidades básicas de um grafo, também chamadas de nós. Cada vértice funciona como um elemento unitário que pode carregar qualquer tipo de valor ou informação. Dependendo da aplicação, um vértice pode representar um espaço na memória, um local, uma ideia ou um estado, como a configuração de um sistema em um dado momento. A definição de grafo não restringe o significado dos vértices, que depende apenas do problema modelado.

2.1.1.2 Arestas

Arestas são as conexões que ligam pares de vértices de um grafo. Cada aresta representa uma relação entre dois vértices, e seu significado depende da aplicação. Uma aresta pode representar uma passagem física entre dois locais, uma passagem lógica entre dois estados ou uma mudança de estado, como a transição de um sistema de uma configuração para outra.

2.1.1.3 Lista de adjacência

Lista de adjacência é uma forma de representar um grafo em que cada vértice armazena a lista dos vértices diretamente conectados a ele por uma aresta. Para um grafo com n vértices e m arestas, essa estrutura ocupa espaço proporcional a $n + m$, o que a torna eficiente para grafos com poucas arestas (Cormen et al., 2009).

2.1.1.4 Matriz de adjacência

Matriz de adjacência é uma forma de representar um grafo por meio de uma tabela quadrada de dimensão $n \times n$, em que cada linha e cada coluna correspondem a um vértice. Cada posição da matriz indica se existe uma aresta entre o par de vértices correspondente, e pode também armazenar o peso dessa aresta. Em um grafo não direcionado, a matriz é simétrica, pois a conexão vale nos dois sentidos.

Essa estrutura ocupa espaço proporcional a n^2, o que a torna pouco eficiente para grafos com poucas arestas. Em compensação, ao contrário da lista de adjacência (Seção 2.1.1.3), ela permite verificar em tempo constante se dois vértices estão conectados (Gomes, 2022).

2.1.2 Direcionamento

Como definido em Seção 2.1.1.2, uma aresta conecta dois vértices. Essa conexão pode ter um sentido, caso em que a aresta é direcionada, ou representar apenas uma ligação sem direção. Na criação de um grafo, é possível defini-lo como direcionado (Figura 3.a) ou não direcionado (Figura 3.b). Ou seja, a conexão entre dois vértices pode ser unidirecional ou bidirecional.

Figura 3 – Exemplo de grafo direcionado e não direcionado.



(a) Grafo direcionado (b) Grafo não direcionado

Fonte: Elaborado pelo autor.

A escolha entre grafos direcionados e não direcionados depende do problema a ser modelado. Em muitos casos, como em mapas de ruas ou redes de transporte, as conexões são bidirecionais, enquanto em outros, como fluxos de dados ou hierarquias, as conexões podem ser unidirecionais.

2.1.3 Pesos

Grafos também podem ser ponderados (Figura 4.a), atribuindo valores numéricos às arestas para representar custos ou distâncias entre os vértices.

Figura 4 – Exemplo de grafo ponderado e não ponderado.



(a) Grafo ponderado (b) Grafo não ponderado

Fonte: Elaborado pelo autor.

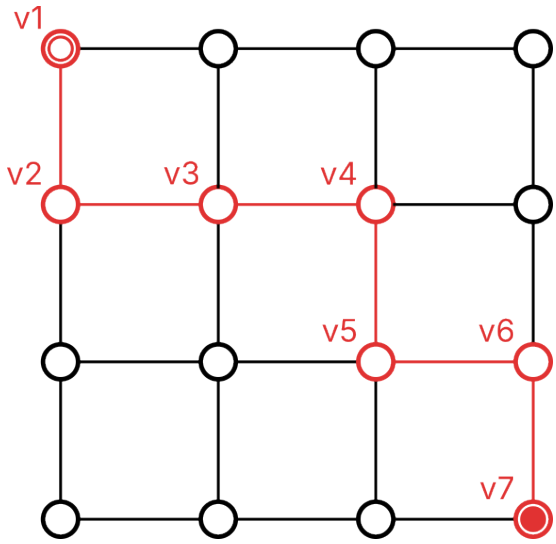
Ponderar a travessia entre vértices permite representar de forma mais precisa as características do ambiente, como a dificuldade de atravessar determinados terrenos ou a distância entre pontos. Como visto na Figura 4, a atribuição de pesos às arestas pode influenciar diretamente a escolha do caminho mais eficiente ou mais curto.

O custo de travessia de uma aresta pode representar diferentes aspectos, como distância física, tempo de viagem ou dificuldade de navegação. A escolha de atribuir pesos às arestas depende do contexto do problema e dos objetivos da estrutura. Alguns algoritmos de busca, como o de Dijkstra, exigem que os pesos sejam não-negativos (Cormen et al., 2009).

2.1.4 Caminho

Um caminho em um grafo é uma sequência de vértices conectados por arestas, representando uma rota entre dois pontos (Gomes, 2022), ilustrado na Figura 5. A definição formal de um caminho é uma sequência de vértices $(v_1, v_2, ..., v_n)$ tal que cada par consecutivo $\{v_i, v_{i+1}\}$ está conectado por uma aresta no grafo.

Figura 5 – Exemplo de caminho em um grafo.



Fonte: Elaborado pelo autor.

Considerando v_1 o vértice inicial e v_7 o vértice final, a sequência de vértices $(v_1, v_2, v_3, v_4, v_5, v_6, v_7)$ representada na Figura 5 é o caminho $v_1 \rightarrow v_7$. Esse caminho é uma das várias rotas possíveis entre os vértices v_1 e v_7, e a escolha do caminho mais eficiente depende dos pesos das arestas e das características do grafo.

Para todo vértice v em um grafo, é possível determinar se existe um caminho para outro vértice u no mesmo grafo. Entretanto, a existência de um caminho não impede a existência de outros caminhos entre v e u, podendo haver múltiplos caminhos com diferentes pesos e características. Quando comparados, os caminhos podem apresentar variações significativas em termos de custo, distância ou dificuldade de travessia. Se existe ao menos um caminho entre v e u, e o grafo não contém ciclos de peso negativo, existe um caminho mínimo ou ótimo entre eles, que é aquele com o menor custo ou peso total.

2.1.5 Componente conexo

Dado um grafo não direcionado, um componente conexo é um subgrafo conexo e maximal, isto é, não pode ser estendido com outros vértices ou arestas do grafo original, e nele existe caminho entre quaisquer pares de vértices (Gomes, 2022).

Figura 6 – Exemplo de componentes conexos em um grafo.

Fonte: Elaborado pelo autor.

A identificação de componentes conexos é fundamental para entender a estrutura do grafo e determinar a viabilidade de encontrar caminhos entre diferentes regiões. Uma vez que um vértice pertence a um componente conexo, é possível garantir que existe um caminho para qualquer outro vértice dentro do mesmo componente.

2.2 Busca

A busca de caminhos em um grafo é o processo de encontrar um caminho entre um vértice inicial e um vértice objetivo, idealmente o de menor custo. Diferentes algoritmos de busca podem ser aplicados, dependendo das características do grafo e dos requisitos do problema.

Figura 7 – Diversos caminhos possíveis entre dois vértices em um grafo.

Fonte: Elaborado pelo autor.

Sabendo da necessidade de encontrar caminhos mínimos, algoritmos de busca são projetados para explorar o grafo de maneiras específicas, visando otimizar a descoberta de caminhos com base em critérios como custo, distância ou heurísticas.

2.2.1 Dijkstra

O algoritmo de Dijkstra, proposto por Dijkstra (1959), é um método clássico da busca de caminhos em grafos ponderados, considerando que todos os pesos das arestas são não-negativos. Ele é capaz de encontrar, para cada vértice alcançável, um caminho mínimo a partir de um vértice inicial (Gomes, 2022). Esse processo gera uma árvore de caminhos mínimos tendo o vértice inicial como raiz.

Na implementação do algoritmo de Dijkstra, é considerado o pseudocódigo apresentado por Gomes (2022). Neste capítulo consideraremos um grafo $G = (V, A)$ ponderado de pesos não-negativos, onde V é o conjunto de vértices e A é o conjunto de arestas.

Figura 8 – Pseudocódigo do algoritmo de Dijkstra.

```
Dijkstra(G, W, s)
1   para cada v ∈ V faça
2       g_v ← inf
3       p_v ← nil
4   d_s ← 0
5   Q ← V
6   S ← ∅
7   enquanto Q ≠ ∅ faça
8       u ← pop(Q)
9       S ← S ∪ {u}
10      para cada vértice v ∈ adj(u) faça
11          se v ∈ Q e [g_v > d_u + W_{u,v}] então
12              g_v ← (d_u + W_{u,v})
13              p_v ← u
```

Fonte: Gomes (2022).

Na Figura 8, o algoritmo de Dijkstra inicializa a distância g_v de todos os vértices a partir do vértice inicial s como infinitos, exceto para o vértice inicial, que tem distância zero para ele mesmo. Em seguida são criadas duas estruturas de dados:

- Q: Uma fila de prioridade mínima (ou *min-PQ*, do inglês *Minimum Priority Queue*) que armazena os vértices ainda não processados ordenados pela menor distância conhecida, de modo que o vértice de menor distância seja sempre o primeiro a sair.
- S: Um conjunto que armazena os vértices cuja distância mínima já foi determinada. Os caminhos são reconstruídos a partir dos predecessores p_v.

O laço principal do algoritmo repete-se enquanto Q não estiver vazia. Em cada iteração, a operação $\mathrm{pop}(Q)$ retorna o vértice u com menor distância estimada d_u e o remove da fila. Em seguida, u é incluído em S. Como os pesos são não-negativos, nenhum caminho que passe por vértices ainda não processados pode reduzir d_u, o que garante que essa distância já é a mínima.

Para cada vértice v adjacente a u (ver Seção 2.1.1.3) que ainda está em Q, verifica-se se passar por u produz um caminho melhor do que o já conhecido. Se $g_v > d_u + W_{u,v}$, atualiza-se g_v para $d_u + W_{u,v}$ e registra-se u como predecessor de v em p_v. Essa operação é chamada de relaxamento da aresta (u, v).

Ao término do laço, para cada vértice alcançável a partir de s, g_v contém a distância mínima entre s e v. Os predecessores p_v permitem reconstruir o caminho mínimo, percorrendo-os do destino até s.

2.2.2 Heurísticas

Uma heurística é uma função $h(n)$ que estima o custo de um caminho entre um vértice n e o objetivo, sem precisar percorrê-lo. Ela orienta a busca para os vértices mais promissores. Uma heurística é admissível quando nunca superestima o custo real até o objetivo (Hart; Nilsson; Raphael, 1968). Já a consistência exige que, para cada aresta, a estimativa do vértice de origem não exceda o custo da aresta somado à estimativa do vértice de destino.

Em algumas variantes, a heurística recebe um fator de ponderação w, e a função de avaliação passa a ser $f(n) = g(n) + w \cdot h(n)$, em que $g(n)$ é o custo acumulado desde o vértice inicial até n. Com $w = 0$, a heurística não influencia a escolha, e a busca se comporta como o Dijkstra. Valores maiores de w tornam a busca mais orientada pela estimativa, o que pode reduzir o número de vértices explorados à custa da otimalidade do caminho (Pohl, 1970).

A forma de calcular $h(n)$ depende do tipo de movimento permitido no espaço de busca. Em terrenos 3D, a estimativa mais comum é a distância euclidiana, que mede a distância em linha reta entre os vértices:

$$h(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2} \tag{2}$$

Em grades com movimentos apenas nos eixos, a distância de Manhattan soma as diferenças em cada coordenada:

$$h(n) = |x_2 - x_1| + |y_2 - y_1| + |z_2 - z_1| \tag{3}$$

A distância de Chebyshev toma a maior diferença entre as coordenadas. Ela corresponde ao número mínimo de passos em uma grade que permite movimentos diagonais, desde que cada passo tenha o mesmo custo:

$$h(n) = \max(|x_2 - x_1|, |y_2 - y_1|, |z_2 - z_1|) \tag{4}$$

A escolha da heurística depende de ela ser compatível com o custo das arestas. A euclidiana é admissível quando nenhuma aresta custa menos que a distância em linha reta entre seus extremos. A Chebyshev é admissível quando os movimentos diagonais custam o mesmo que os movimentos retos. Já a Manhattan superestima a distância real quando há movimentos diagonais, o que pode levar a caminhos não ótimos.

Como a heurística define a ordem de exploração, ela altera tanto o número de vértices visitados quanto a qualidade do caminho encontrado. Uma heurística admissível garante o caminho ótimo, enquanto uma heurística que superestima o custo pode explorar menos vértices e, ainda assim, não encontrar o menor caminho.

2.2.3 A*

O A* é um algoritmo de busca que combina o custo acumulado com a estimativa do custo restante. Para cada vértice n, ele avalia

$$f(n) = g(n) + h(n) \tag{5}$$

em que $g(n)$ é o custo do caminho desde o vértice inicial até n e $h(n)$ é a heurística de n até o objetivo. Como no Dijkstra, a busca usa uma fila de prioridade, mas ordena os vértices por $f(n)$, e não apenas por $g(n)$. Se a heurística for admissível, o caminho retornado tem o menor custo possível (Hart; Nilsson; Raphael, 1968).

Figura 9 – Pseudocódigo do algoritmo A*.

```
A*(G, w, s, t, h)
1   para cada v ∈ V faça
2       g_v ← inf
3       p_v ← nil
4   g_s ← 0
5   Q ← {s}
6   S ← ∅
7   enquanto Q ≠ ∅ faça
8       # vértice de menor f(u) = g_u + h(u)
9       u ← pop(Q)
10      se u = t então retorna o caminho até t por p
11      S ← S ∪ {u}
12      para cada vértice v ∈ adj(u) faça
13          se v ∉ S e [g_v > g_u + W_{u,v}] então
14              g_v ← (g_u + W_{u,v})
15              p_v ← u
16              se v ∉ Q então Q ← Q ∪ {v}
```

Fonte: Elaborado pelo autor, com base em Gomes (2022) e Hart; Nilsson; Raphael (1968).

Na Figura 9, as diferenças em relação ao Dijkstra estão em negrito. A inicialização das distâncias é igual à do Dijkstra, mas a fila Q começa apenas com o vértice inicial s, pois a busca só precisa explorar a fronteira a partir dele.

A principal diferença está na escolha do vértice a ser expandido. Enquanto o Dijkstra remove de Q o vértice de menor distância, o A* remove o vértice de menor $f(u)$. Quando $h(n) = 0$ para todo n, as duas escolhas coincidem, e o Dijkstra passa a ser um caso particular do A*.

Outra diferença é o critério de parada. O algoritmo termina quando o vértice removido é o objetivo t, e não quando Q esvazia. Essa parada antecipada pode reduzir o número de vértices explorados. Vértices já finalizados em S não são mais atualizados, e um vértice entra em Q na primeira vez que é alcançado.

2.2.4 Weighted A*

O Weighted A* é uma variante do A* em que a heurística é multiplicada por um fator de ponderação w. A função de avaliação passa a ser:

$$f(n) = g(n) + w \cdot h(n) \tag{6}$$

Com $w = 1$, a busca coincide com o A*. Com $w = 0$, a heurística deixa de influenciar a escolha e o algoritmo se comporta como o Dijkstra. Para $w > 1$, a estimativa passa a pesar mais na escolha, o que tende a reduzir o número de vértices explorados. Em contrapartida, o caminho encontrado pode não ser o de menor custo (Pohl, 1970).

Figura 10 – Pseudocódigo do algoritmo Weighted A*.

```
Weighted A*(G, W, s, t, h)
1   para cada v ∈ V faça
2       g_v ← inf
3       p_v ← nil
4   g_s ← 0
5   Q ← {s}
6   S ← ∅
7   enquanto Q ≠ ∅ faça
8       # vértice de menor f(u) = g_u + * w * · h(u)
9       u ← pop(Q)
10      se u = t então retorna o caminho até t por p
11      S ← S ∪ {u}
12      para cada vértice v ∈ adj(u) faça
13          se v ∉ S e [g_v > g_u + W_{u,v}] então
14              g_v ← (g_u + W_{u,v})
15              p_v ← u
16              se v ∉ Q então Q ← Q ∪ {v}
```

Fonte: Elaborado pelo autor, com base em Gomes (2022) e Hart; Nilsson; Raphael (1968).

Na Figura 10, as diferenças em relação ao Dijkstra estão em negrito, e são as mesmas do A*, acrescidas do fator w na seleção do vértice. Como a estrutura do pseudocódigo é a do A*, a única mudança de comportamento vem da função de avaliação, e por isso o restante da explicação do A* se aplica aqui.

O valor de w define o equilíbrio entre velocidade e otimalidade. Valores próximos de 1 mantêm a garantia do A* quando a heurística é admissível. Valores maiores fazem a busca se aproximar do comportamento de uma busca gulosa, que é o caso tratado a seguir.

2.2.5 Greedy Best-First Search

A Greedy Best-First Search ordena os vértices apenas pela heurística, sem considerar o custo acumulado até eles. Assim, a função de avaliação é:

$$f(n) = h(n) \tag{7}$$

Essa escolha torna a busca rápida, porque ela tende a seguir diretamente na direção do objetivo. Em contrapartida, como o custo acumulado é ignorado, o caminho encontrado pode ser longo e não é garantidamente o de menor custo. A Greedy Best--First Search corresponde ao caso em que o peso da heurística tende ao infinito no Weighted A*.

Figura 11 – Pseudocódigo da Greedy Best-First Search.

```
Greedy Best-First Search(G, s, t, h)
para cada v ∈ V faça
    p_v ← nil
Q ← {s}
S ← ∅
enquanto Q ≠ ∅ faça
    # vértice de menor h(u)
    u ← pop(Q)
    se u = t então retorna o caminho até t por p
    S ← S ∪ {u}
    para cada vértice v ∈ adj(u) faça
        se v ∉ S e v ∉ Q então
            p_v ← u
            Q ← Q ∪ {v}
```

Fonte: Elaborado pelo autor, com base em Gomes (2022) e Hart; Nilsson; Raphael (1968).

Na Figura 11, as diferenças em relação ao Dijkstra estão em negrito. Não há g_v nem relaxação: a fila é ordenada apenas por $h(u)$, e cada vértice é inserido uma única vez, na primeira vez em que é descoberto. Por isso, a inicialização não precisa das distâncias, e a condição de atualização depende apenas de o vértice já ter sido visitado ou estar na fila.

2.3 Geração Procedural de Terrenos

A geração procedural consiste em criar conteúdo automaticamente, a partir de algoritmos e de um conjunto de regras ou parâmetros. Em terrenos 3D, essa abordagem produz paisagens complexas e variadas sem modelagem manual. Como o resultado depende apenas dos parâmetros e de uma semente inicial, a mesma configuração pode ser reproduzida com exatidão.

2.3.1 Mapas de altura

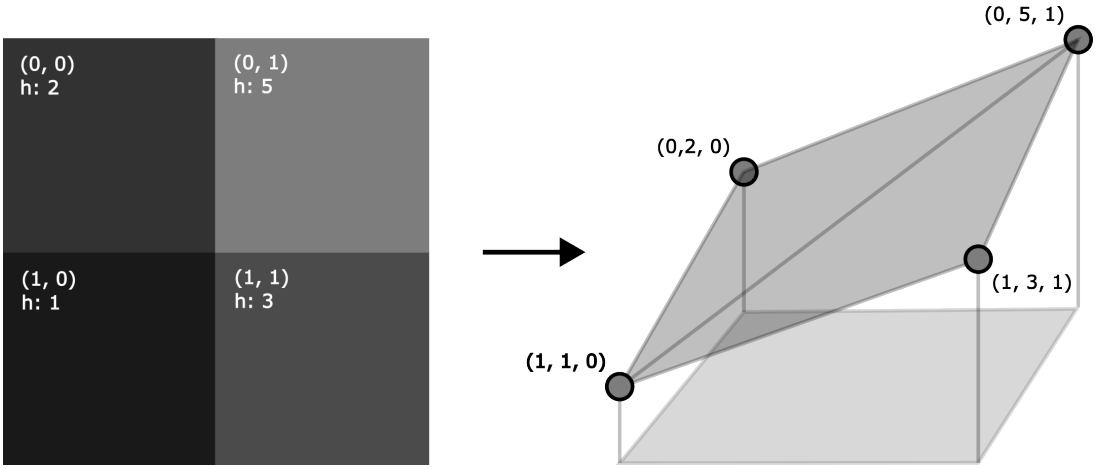
Um mapa de altura é uma matriz bidimensional em que cada elemento armazena a altura do terreno naquele ponto. Quando essa matriz é exibida como imagem em escala de cinza, a intensidade de cada pixel corresponde à elevação. Em geral, os valores são normalizados no intervalo $[0, 1]$ e multiplicados por um fator de escala H, que define a altura máxima do terreno.

Para obter a geometria tridimensional, cada elemento (i, j) da matriz é associado a um vértice cujas coordenadas são

$$(x, y, z) = (i, h(i, j), j) \tag{8}$$

em que $h(i, j)$ é a altura armazenada naquele ponto. Dessa forma, o plano horizontal da grade é preservado, e a altura passa a ser a coordenada y (Figura 12).

Figura 12 – Conversão de um mapa de altura em vértices tridimensionais.



Fonte: Elaborado pelo autor.

Dessa forma, um pixel totalmente preto ($v = 0$) resultará em uma altura de 0, enquanto um pixel totalmente branco ($v = 255$) atingirá a altura máxima estipulada (H), permitindo um controle preciso sobre a amplitude do relevo gerado.

Por fim, são criadas faces conectando cada vértice com seus vizinhos diretos, formando uma malha de triângulos que representa a superfície do terreno. Essa malha pode então ser utilizada como cenário de teste para os algoritmos de *pathfinding*, permitindo avaliar seu desempenho em ambientes tridimensionais complexos.

Figura 13 – Exemplo de malha gerada a partir de um mapa de altura



Fonte: Elaborado pelo autor.

A criação de mapas de altura pode ser feita a partir de imagens reais, como fotografias aéreas ou imagens de satélite, ou gerada proceduralmente por algoritmos de ruído, como o ruído de Perlin (Seção 2.3.2).

2.3.2 Ruído de Perlin

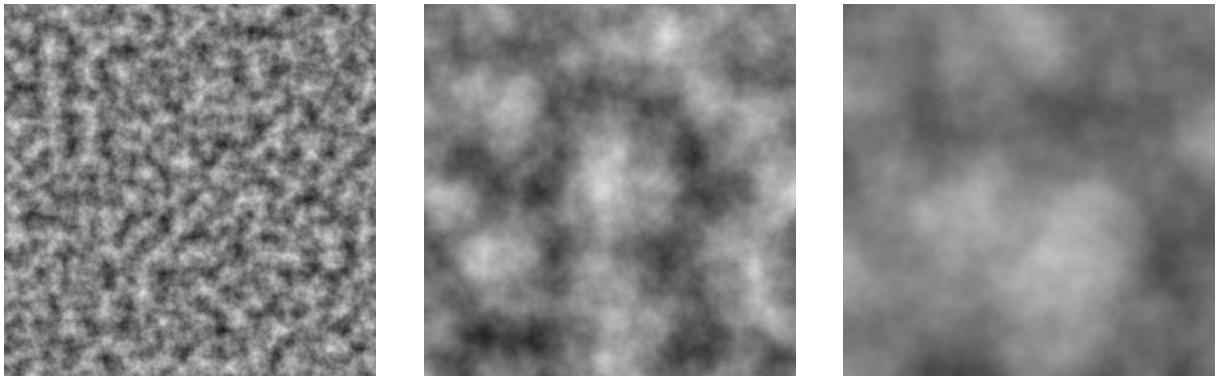
O ruído de Perlin é um algoritmo de geração de ruído gradiente, proposto por Perlin (1985), que produz variações contínuas e suaves. Diferentemente de um ruído puramente aleatório, cada valor depende de uma grade regular de vetores de gradiente, o que faz o resultado variar gradualmente. Por isso, o ruído de Perlin é adequado para simular relevos naturais.

O valor do ruído em um ponto é obtido a partir da célula da grade que o contém. Cada canto c dessa célula possui um vetor de gradiente unitário $\vec{g}_c$, com direção aleatória. Para cada canto, calcula-se o vetor de distância $\vec{d}_c$ entre o ponto e o canto, e o produto escalar entre os dois vetores:

$$s_c = \vec{d}_c \cdot \vec{g}_c = d_{c,x} g_{c,x} + d_{c,y} g_{c,y} \tag{9}$$

Esses valores s_c são as contribuições de cada canto. O tamanho da célula determina o nível de detalhe: células maiores produzem um ruído mais suave, e células menores produzem um ruído mais detalhado.

Figura 14 – Ruído de Perlin com células de diferentes tamanhos.



(a) Ruído gerado com células pequenas.

(b) Ruído gerado com células medianas.

(c) Ruído gerado com células grandes.

Fonte: Elaborado pelo autor.

2.3.2.1 Função de suavização

As contribuições dos cantos são combinadas por interpolação, dentro da própria célula. A interpolação linear simples produz transições com mudança abrupta de inclinação nas bordas da célula, o que se torna visível no terreno. Para evitar isso, o peso de interpolação w é substituído por uma função de suavização $S(w)$. A proposta

original de Perlin utiliza uma curva cúbica, e versões posteriores empregam a curva quíntica:

$$S(w) = 6w^5 - 15w^4 + 10w^3 \tag{10}$$

Com essa função, a interpolação entre dois valores a_0 e a_1 é dada por:

$$f(a_0, a_1, w) = a_0 + (a_1 - a_0) \cdot S(w) \tag{11}$$

2.3.2.2 Oitavas

Uma única oitava do ruído tem um nível de detalhe fixo. Para obter relevos com variações em diferentes escalas, somam-se várias oitavas, cada uma com frequência e amplitude próprias. Para a oitava i, com $i = 0, 1, ..., N-1$, tem-se

$$f_i = f \cdot l^i, \quad a_i = a_0 \cdot p^i \tag{12}$$

e o valor final do ruído é a soma das oitavas:

$$V(x, y) = \sum_{i=0}^{N-1} a_i \cdot \text{ruído}(x \cdot f_i, y \cdot f_i) \tag{13}$$

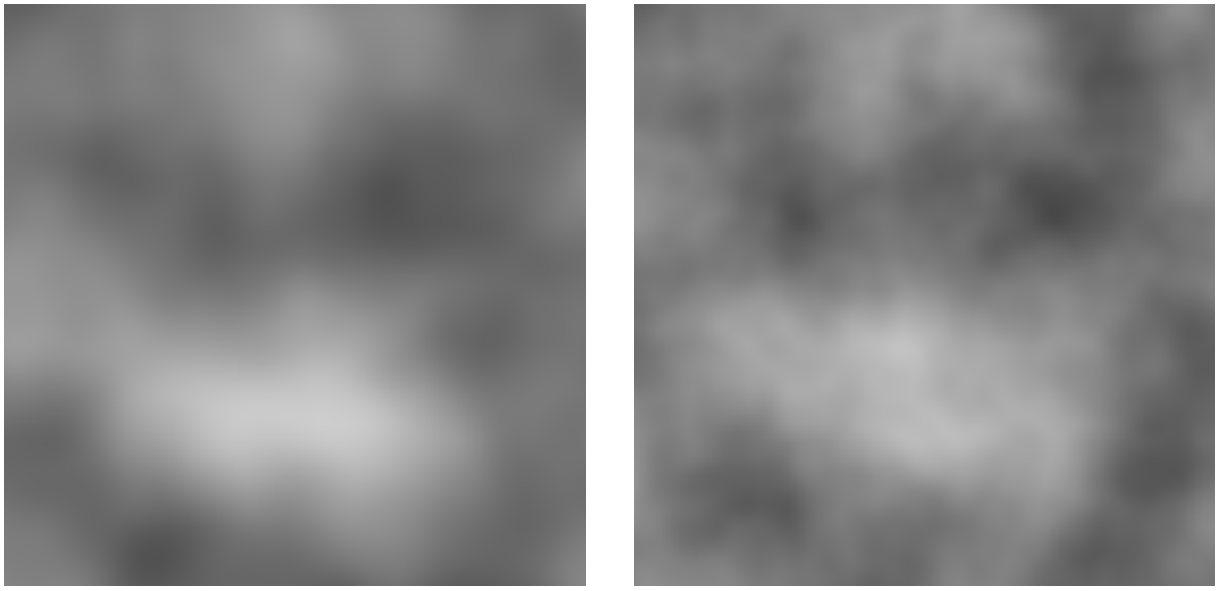
Os parâmetros f, a_0, l, p e N controlam o resultado final de maneiras distintas, descritas a seguir.

A frequência inicial f define a escala do ruído na primeira oitava. Quando f aumenta, o padrão se repete com mais proximidade, e uma mesma área apresenta mais elevações e depressões. Como todas as oitavas são multiplicadas pela mesma proporção de f, a escala de todo o terreno muda, e não apenas um detalhe.

A amplitude inicial a_0 define o alcance vertical da primeira oitava. Como $a_i = a_0 \cdot p^i$, aumentar a_0 multiplica a contribuição de todas as oitavas pelo mesmo fator. Assim, a_0 altera a altura total do relevo sem mudar a proporção entre detalhes grandes e pequenos.

A lacunaridade l determina quanto a frequência cresce de uma oitava para a seguinte. Valores altos fazem as oitavas finais ter frequências muito maiores que as iniciais, de modo que o detalhe se concentra em escalas pequenas e o relevo fica mais irregular. Valores próximos de 1 fazem todas as oitavas ter frequência semelhante, e o resultado se aproxima de um único ruído escalado. A Figura 15 compara ruídos com lacunaridade baixa e alta, mantendo os demais parâmetros.

Figura 15 – Variação da lacunaridade: ruídos com lacunaridade baixa e alta.

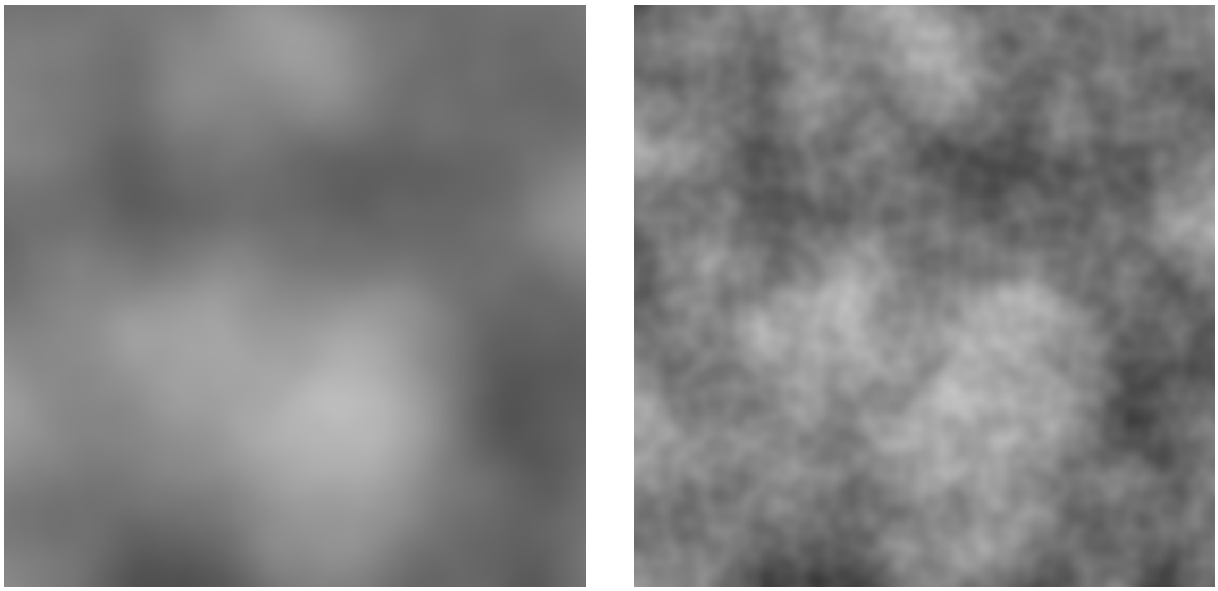


(a) Ruído de baixa lacunaridade. (b) Ruído de alta lacunaridade.

Fonte: Elaborado pelo autor.

A persistência p determina quanto a amplitude diminui de uma oitava para a seguinte. Valores baixos fazem os detalhes finos ter pouca influência, e o relevo fica dominado pelas formas amplas. Valores altos mantêm as oitavas finas com peso comparável ao das oitavas grandes, e o terreno fica mais áspero. A Figura 16 mostra essa diferença, mantendo os demais parâmetros.

Figura 16 – Variação da persistência: ruídos com persistência baixa e alta.

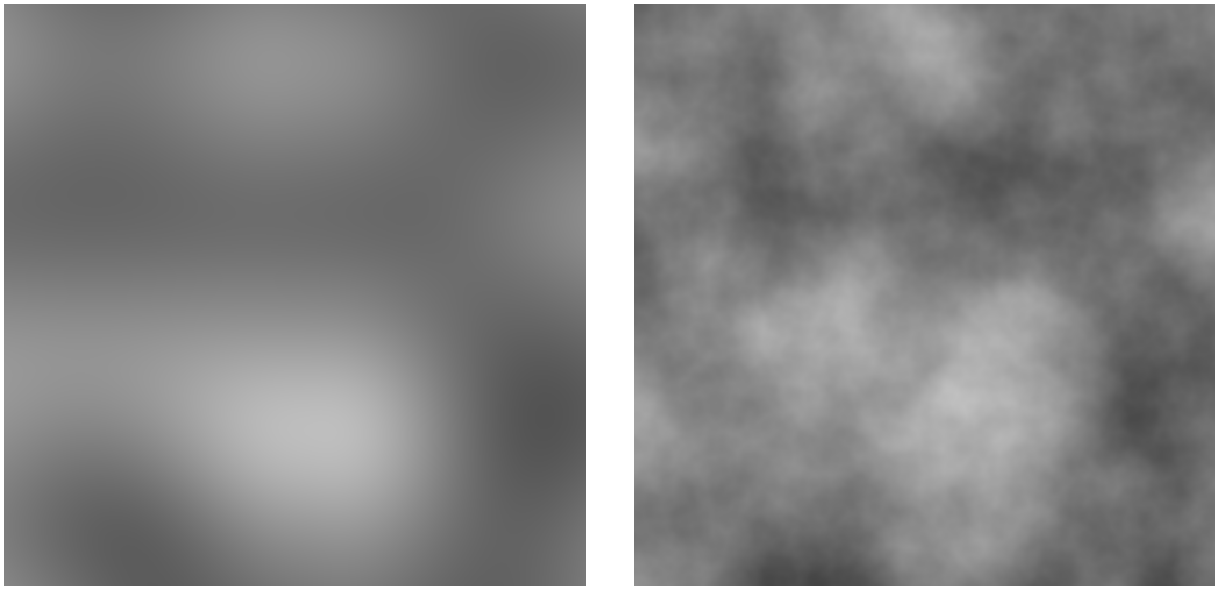


(a) Ruído de baixa amplitude/persistência. (b) Ruído de alta amplitude/persistência.

Fonte: Elaborado pelo autor.

O número de oitavas N define quantas camadas são somadas. Com poucas oitavas, o ruído contém apenas as escalas mais grosseiras. Cada oitava adicional acrescenta detalhes menores, cujo peso é reduzido por p^i. Por isso, com persistência baixa, oitavas adicionais quase não alteram o resultado, e com persistência alta, cada uma contribui de forma perceptível (Figura 17).

Figura 17 – Variação do número de oitavas: ruídos com 1 e 6 oitavas.



(a) Ruído com 1 oitava. (b) Ruído com 6 oitavas.

Fonte: Elaborado pelo autor.

Valores comumente usados são próximos de 2 para a lacunaridade e de 0,5 para a persistência, de modo que cada oitava tem o dobro da frequência e metade da amplitude da anterior. Essa combinação é descrita por Patel (2015), e produz terrenos com grandes elevações e pequenas variações ao mesmo tempo.

2.4 Representações Geométricas de Terrenos

Esta seção descreve formas de representar a geometria visual de terrenos em um ambiente 3D. Cada técnica define como o volume ou a superfície do terreno é armazenado, e essa escolha determina quais formas podem ser descritas.

2.4.1 Voxels

O *voxel* é a unidade cúbica que divide o espaço tridimensional em uma grade regular, de modo análogo ao pixel em imagens bidimensionais. Em uma representação volumétrica de terreno, cada *voxel* pode indicar se a posição correspondente está preenchida por material ou vazia. Essa abordagem descreve cavidades e estruturas sobrepostas, que um mapa de altura não armazena, já que ele guarda apenas uma altura para cada posição horizontal.

2.4.2 Marching Cubes/Transvoxel

A extração de uma superfície a partir de um campo volumétrico pode ser feita pelo algoritmo de Marching Cubes, proposto por Lorensen; Cline (1987). Para cada célula do volume, o algoritmo compara os valores dos oito cantos com um limiar, classifica cada canto como acima ou abaixo dele e gera os triângulos correspondentes à configuração resultante. Como essa classificação produz 256 configurações possíveis, o algoritmo usa tabelas de casos para determinar os triângulos de cada uma.

O Transvoxel, apresentado por Lengyel (2010), estende essa abordagem a terrenos baseados em voxels em tempo real. Ele acrescenta tabelas de casos para células de transição entre níveis de detalhe distintos, o que evita rachaduras na superfície nas fronteiras entre blocos de resoluções diferentes.

Figura 18 – Exemplo de reconstrução de superfícies com Marching Cubes e Transvoxel.

Fonte: Elaborado pelo autor.

2.5 Representações Navegáveis de Terrenos

A busca de caminhos depende de uma estrutura que represente as regiões por onde um agente pode se deslocar. Essa estrutura é uma representação navegável, em geral um grafo $G = (V, A)$ (ver Seção 2.1), obtido a partir da geometria do terreno. Seus vértices correspondem a posições ou regiões transitáveis, e suas arestas indicam a passagem entre elas. Diferentes representações partem do mesmo terreno, mas produzem grafos com quantidades distintas de vértices e arestas. Conforme Algfoor; Sunar; Kolivand (2015), o *pathfinding* pode ser dividido em duas etapas: a geração do grafo e o algoritmo de busca.

2.5.1 Grade 2.5D

A grade 2.5D é uma matriz bidimensional de alturas, em que cada célula corresponde a uma posição do plano horizontal e armazena um único valor de altura. Como grafo, ela corresponde ao mapa de altura (ver Seção 2.3.1): cada célula é um vértice, ligado às células vizinhas, e a existência de cada aresta pode depender de um critério de transitabilidade, como a diferença de altura entre as células. Por armazenar uma altura por posição, a grade descreve a superfície do terreno, mas não representa cavidades nem estruturas sobrepostas.

Figura 19 – Exemplo de grade 2.5D.

Fonte: Elaborado pelo autor.

2.5.2 Superfície Voxel

Na superfície voxel, a representação volumétrica (ver Seção 2.4.1) fornece os vértices do grafo: cada vértice corresponde a um *voxel* da superfície, e as arestas ligam *voxels* vizinhos na grade tridimensional. Diferentemente da grade 2.5D, essa estrutura registra altura e volume, o que permite representar regiões sobrepostas do terreno.

Figura 20 – Exemplo de superfície voxel.

Fonte: Elaborado pelo autor.

2.5.3 Grafo de Vértices

No grafo de vértices, cada vértice da malha poligonal do terreno dá origem a um vértice navegável, e cada aresta liga dois vértices que são extremidades de uma mesma aresta da malha. A estrutura reaproveita a topologia da malha, sem criar vértices novos.

Figura 21 – Exemplo de grafo de vértices.

Fonte: Elaborado pelo autor.

2.5.4 Grafo Dual

No grafo dual, cada polígono da malha corresponde a um vértice, que representa uma região navegável, e dois vértices são ligados por uma aresta quando os polígonos correspondentes compartilham uma aresta da malha. O número de vértices do grafo é, portanto, o número de polígonos da malha, e não o número de seus vértices. Essa construção é a base das malhas de navegação (*navmesh*), nas quais os polígonos são regiões caminháveis e a adjacência entre eles forma o grafo, conforme Snook (2000). A mesma ideia de adjacência entre triângulos é empregada na busca em triangulações, como em Demyen; Buro (2006).

Figura 22 – Exemplo de grafo dual.

Fonte: Elaborado pelo autor.

2.6 Avaliação de desempenho e estatísticas

ANOVA e post-hoc de Tukey.

2.7 Trabalhos relacionados

2.7.1 Johansson

2.7.2 Kapi

2.7.3 Matriz de diferença

figura: matrix de diferença

3 DESENVOLVIMENTO

Neste capítulo, são apresentados os detalhes do desenvolvimento do trabalho, incluindo a arquitetura do sistema, a implementação dos algoritmos de busca, a geração de terrenos e a instrumentação para coleta de dados e testes.

3.1 Pipeline

Ruido -> Heightmap -> Representação Navegável -> Algoritmo de Busca -> Coleta de métricas -> Análise estatística

figura: pipeline do desenvolvimento

3.2 IFCG e visualizador concorrente

Motor gráfico para visualização de terrenos e caminhos encontrados pelos algoritmos de busca, permitindo a inspeção visual dos resultados.

figura: cena renderizada e diagrama taskmaster

3.3 IwGraph<T>

Estrutura de dados genérica para representar grafos, utilizada na implementação dos algoritmos de busca e na representação navegável dos terrenos.

3.4 search.hpp

Algoritmos de busca implementados em C++, incluindo Dijkstra, A*, WA* e GBFS, com suporte a aquisição de dados e coleta de métricas.

3.5 Gerador de terrenos

Ferramenta para geração de terrenos 3D, permitindo a criação de modelos realistas com diferentes características e propriedades.

figura: exemplo de diferentes representações navegaveis do mesmo terreno

3.6 GraphModel, GraphFactory, isTranversable

Classes para modelar, fabricar e verificar a traversabilidade de grafos.

3.7 Instrumentação

Ferramentas e técnicas para coleta de dados e testes, incluindo aquisição de métricas e análise estatística.

3.7.1 Statistics

Classe para coletar e exportar métricas.

3.7.2 acquire.sh

Script para aquisição de métricas, executando os algoritmos de busca em diferentes representações navegáveis de terrenos e coletando os resultados.

3.7.3 CLI

Interface de linha de comando para executar os testes, coletas ou exportações de dados.

4 METODOLOGIA

Neste capítulo, são descritos os métodos e procedimentos utilizados para conduzir a pesquisa, incluindo o ambiente experimental, as variáveis analisadas, os níveis de teste, o procedimento de coleta de dados e as métricas utilizadas para avaliação dos resultados.

4.1 Tipo de pesquisa

Definição do tipo de pesquisa realizada, incluindo a abordagem experimental, comparativa e estatística adotada para avaliar o desempenho dos algoritmos de busca em diferentes representações navegáveis de terrenos 3D.

4.2 Ambiente

Configuração do ambiente experimental, incluindo hardware, software, bibliotecas e ferramentas utilizadas para a implementação dos algoritmos de busca, geração de terrenos e coleta de métricas.

4.3 Variáveis

Variáveis independentes e dependentes analisadas na pesquisa, incluindo as representações navegáveis de terrenos, os algoritmos de busca e as métricas de desempenho coletadas.

figura: tabela de variáveis independentes e dependentes

4.4 Níveis

Definição dos níveis de teste para cada variável, incluindo diferentes representações navegáveis de terrenos, algoritmos de busca e configurações de parâmetros utilizados nos experimentos.

figura: tabela de níveis de teste para cada variável

4.5 Procedimento e métricas

Descrição do procedimento de coleta de dados e das métricas utilizadas para avaliar o desempenho dos algoritmos de busca.

4.6 Protocolo estatístico

Descrição do protocolo estatístico utilizado para análise dos dados coletados, incluindo testes de hipótese, intervalos de confiança e modelos de regressão.

4.7 Ameaças

Identificação e análise das ameaças à validade interna e externa do estudo, incluindo possíveis fontes de viés, limitações do método e impacto sobre os resultados obtidos.

5 RESULTADOS E DISCUSSÃO

6 CONCLUSÃO

REFERÊNCIAS

ALGFOOR, Zeyad Abd; SUNAR, Mohd Shahrizal; KOLIVAND, Hoshang. A Comprehensive Study on Pathfinding Techniques for Robotics and Video Games. **International Journal of Computer Games Technology**, v. 2015, p. 1–11, 2015.

CORMEN, Thomas H. *et al*. **Introduction to Algorithms**. 3. ed. Cambridge, MA: MIT Press, 2009.

DEMYEN, Douglas; BURO, Michael. Efficient Triangulation-Based Pathfinding. *In:* AAAI Press, 2006.

DIJKSTRA, Edsger W. A Note on Two Problems in Connexion with Graphs. **Numerische Mathematik**, v. 1, n. 1, p. 269–271, 1959.

GOMES, Paulo César Rodacki. **Grafos: conceitos fundamentais, algoritmos e aplicações**. Blumenau, SC: Editora do Instituto Federal Catarinense, 2022.

HART, Peter E. ; NILSSON, Nils J. ; RAPHAEL, Bertram. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. **IEEE Transactions on Systems Science and Cybernetics**, v. 4, n. 2, p. 100–107, 1968.

LENGYEL, Eric. **Voxel-Based Terrain for Real-Time Virtual Simulations**. Doctoral dissertation—*[S.l.: S.n.]*.

LORENSEN, William E.; CLINE, Harvey E. Marching Cubes: A High Resolution 3D Surface Construction Algorithm. *In:* : SIGGRAPH '87.New York, NY, USA: Association for Computing Machinery, 1987.

PARDEDE, Sara Lutami *et al*. A review of pathfinding in game development. **Journal of Computer Engineering**, v. 1, n. 01, p. 47–56, 2022.

PATEL, Amit J. **Making maps with noise functions**. Disponível em: <https://www.redblobgames.com/maps/terrain-from-noise/#elevation-redistribution>. Acesso em: 7 maio. 2026.

PERLIN, Ken. An Image Synthesizer. *In:* : SIGGRAPH '85.New York, NY, USA: Association for Computing Machinery, 1985. Disponível em: <https://doi.org/10.1145/325334.325247>

POHL, Ira. Heuristic Search Viewed as Path Finding in a Graph. **Artificial Intelligence**, v. 1, n. 3–4, p. 193–204, 1970.

RUSSELL, Stuart; NORVIG, Peter. **Inteligência Artificial: uma abordagem moderna**. 3. ed. Rio de Janeiro: Elsevier, 2013.

SNOOK, Greg. Simplified 3D Movement and Pathfinding Using Navigation Meshes. *In:* DELOURA, Mark (Org.). **Game Programming Gems**. Rockland, MA: Charles River Media, 2000. p. 288–304.